Assembly Language for x86 Processors

7th Edition , Global Edition

Kip R. Irvine

Chapter 17: Expert MS-DOS Programming

(c) Pearson Education, 2015. All rights reserved. You may modify and copy this slide show for your personal use, or for use in the classroom, as long as this copyright statement, the author's name, and the title are not changed.

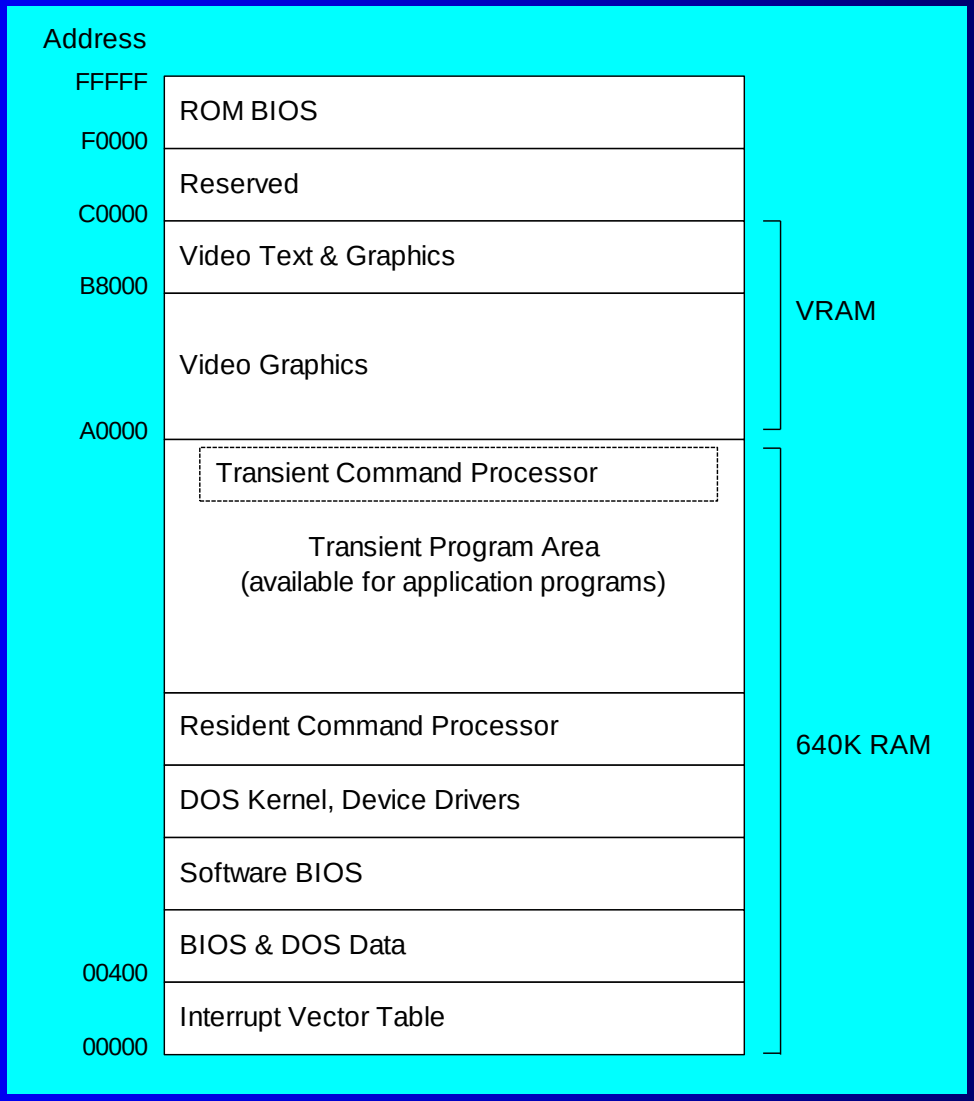
Interrupt Handling

- Overview
- Hardware Interrupts
- Interrupt Control Instructions
- Writing a Custom Interrupt Handler
- Terminate and Stay Resident Programs
- The No_Reset Program

Overview

- Interrupt handler (interrupt service routine) – performs common I/O tasks
  - can be called as functions
  - can be activated by hardware events
- Examples:
  - video output handler
  - critical error handler
  - keyboard handler
  - divide by zero handler
  - Ctrl-Break handler
  - serial port I/O

(Recap) MS-DOS Memory Map



Irvine, Kip R. Assembly Language for x86 Processors 7/e, 2015.

Interrupt Vector Table

- Each entry contains a 32-bit segment/offset address that points to an interrupt service routine
- Offset = *interruptNumber* * 4
- The following are only examples:

Interrupt Number	Offset	Interrupt Vectors
00-03	0000	02C1:5186 0070:0C67 0DAD:2C1B 0070:0C67
04-07	0010	0070:0C67 F000:FF54 F000:837B F000:837B
08-0B	0020	0D70:022C 0DAD:2BAD 0070:0325 0070:039F
0C-0F	0030	0070:0419 0070:0493 0070:050D 0070:0C67
10-13	0040	C000:0CD7 F000:F84D F000:F841 0070:237D

Hardware Interrupts

- Generated by the Intel 8259 Programmable Interrupt Contoller (PIC)
  - in response to a hardware signal
- Interrupt Request Levels (IRQ)
  - priority-based interrupt scheduler
  - brokers simultaneous interrupt requests
  - prevents low-priority interrupt from interrupting a high-priority interrupt

PC Interrupts

D.2 PC Interrupts

Table D-1 General List of PC Interrupt Numbers.[a]

Number	Description
0	*Divide Error*. CPU-generated: activated when attempting to divide by zero.
1	*Single Step*. CPU-generated: active when the CPU Trap flag is set.
2	*Nonmaskable Interrupt*. External hardware: activated when a memory error occurs.
3	*Breakpoint*. CPU-generated: activated when the 0CCh (INT 3) instruction is executed.
4	*INTO Detected Overflow*. CPU-generated: Activated when the INTO instruction is executed and the Overflow flag is set.
5	*Print Screen*. Activated either by the INT 5 instruction or pressing the Shift-PrtSc keys.
6	*Invalid OpCode* (80286+)
7	*Processor Extension Not Available* (80286+)
8	IRQ0: *System Timer Interrupt*. Updates the BIOS clock 18.2 times per second. For your own programming, see INT 1Ch.
9	IRQ1: *Keyboard Hardware Interrupt*. Activated when a key is pressed. Reads the key from the keyboard port and stores it in the keyboard typeahead buffer.
0A	IRQ2: *Programmable Interrupt Controller*
0B	IRQ3: Serial Communications (COM2)
0C	IRQ4: Serial Communications (COM1)
0D	IRQ5: Fixed Disk
0E	IRQ6: *Diskette Interrupt*. Activated when a disk seek is in progress.
0F	IRQ7: *Parallel Printer*

PC Interrupts (cont.)

Table D-1 *(Continued)*

Number	Description
19	*Bootstrap Loader.* Reboot MS-DOS.
1A	*Time of Day.* Get the number of timer ticks since the machine was turned on, or set the counter to a new value. Ticks occur 18.2 times per second.
1B	*Keyboard Break*. This interrupt handler is executed by INT 9h when CTRL-BREAK is pressed.
1C	*User Timer Interrupt*. Empty routine, executed 18.2 times per second. May be used by your own program.
1D	*Video Parameters*. Point to a table containing initialization and information for the video controller chip.
1E	*Diskette Parameters*. Point to a table containing initialization information for the diskette controller.
1F	*Graphics Table*. 8 × 8 Graphics font. Table kept in memory of all extended graphics characters with ASCII codes higher than 127.
20	*Terminate Program*. Terminate a COM program (INT 21h Function 4Ch should be used instead).
21	*MS-DOS Services* (see the complete list in Table D-2).
22	*MS-DOS Terminate Address*. Point to the address of the parent program or process. When the current program ends, this will be the return address.

PC Interrupts (cont.)

23	*MS-DOS Break Address*. MS-DOS jumps here when CTRL-BREAK is pressed.
24	*MS-DOS Critical Error Address*. DOS jumps to this address when there is a critical error in the current program, such as a disk media error.
25	*Absolute Disk Read* (obsolete).
26	*Absolute Disk Write* (obsolete).
27	*Terminate and Stay Resident* (obsolete).
28–FF	(Reserved)
33	*Microsoft Mouse*. Functions that track and control the mouse.
34–3E	*Floating-Point Emulation*.
3F	*Overlay manager*.
40–41	*Fixed Disk Services*. Fixed disk controller.
42–5F	Reserved: specialized uses
60–6B	Available for application programs to use.
6C–7F	Reserved: specialized uses
80–F0	Reserved: used by ROM BASIC.
F1–FF	Available for application programs.

[a]Sources: Ray Duncan, *Advanced MS-DOS*, 2nd ed., Microsoft Press, 1998. *Ralf Brown's Interrupt List*, available on the Web.

INT 21h Appendix

D.3 Interrupt 21H Functions (MS-DOS Services)

There are so many MS-DOS services available through INT 21h that we could not possibly document them all here. Instead, Table D-2 is simply a brief overview of functions that are commonly used.

Table D-2 Interrupt 21h Functions (MS-DOS Services).

Function	Description
1	*Read character from standard input*. If no character is ready, wait for input. Returns: AL = character.
2	*Write character to standard output*. Receives: DL = character.
3	*Read character from standard auxiliary input* (serial port).
4	*Write character to standard auxiliary output* (serial port).
5	*Write character to printer*. Receives: DL = character.
6	*Direct console input/output*. If DL = FFh, read a waiting character from standard input. If DL is any other value, write the character in DL to standard output.
7	*Direct character input without echo*. Wait for a character from standard input. Returns: AL = character.
8	*Character input without echo*. Wait for a character from the standard input device. Returns: AL = character. Character not echoed. May be terminated by Ctrl-Break.
9	*Write string to standard output*. Receives: DS:DX = address of string.
0A	*Buffered keyboard input*. Read a string of characters from the standard input device. Receives: DS:DX points to a predefined keyboard structure.

INT 21h Appendix (cont.)

0B	*Check standard input status*. Check to see if an input character is waiting. Returns: AL = 0FFh if the character is ready; otherwise, AL = 0.
0C	*Clear keyboard buffer and invoke input function*. Clear the console input buffer, and then execute an input function. Receives: AL = desired function (1, 6, 7, 8, or 0Ah).
0E	*Select default drive*. Receives: DL = drive number (0 = A, 1 = B, etc.).
0F–18	FCB file functions (obsolete).
19	*Get current default drive*. Returns: AL = drive number (0 = A, 1 = B, etc.)
1A	*Set disk transfer address*. Receives: DS:DX contains address of disk transfer area.
25	*Set interrupt vector*. Set an entry in the Interrupt Vector Table to a new address. Receives: DS:DX points to the interrupt-handling routine that is inserted in the table; AL = the interrupt number.
26	*Create new program segment prefix*. Receives: DX = segment address for new PSP.
27–29	FCB file functions (obsolete).
2A	*Get system date*. Returns: AL = Day of the week (0–6, where Sunday = 0), CX = year, DH = month, and DL = day.
2B	*Set system date*. Receives: CX = year, DH = month, and DL = day. Returns: AL = 0 if the date is valid.
2C	*Get system time*. Returns: CH = hour, CL = minutes, DH = seconds, and DL = hundredths of seconds.
2D	*Set system time*. Receives: CH = hour, CL = minutes, DH = seconds, and DL = hundredths of seconds. Returns: AL = 0 if the time is valid.
2E	*Set Verify flag*. Receives: AL = new state of MS-DOS Verify flag (0 = off, 1 = on), DL = 00h.

INT 21h Appendix (cont.)

Table D-2 *(Continued)*

Function	Description
2F	*Get disk transfer address* (DTA). Returns: ES:BX = address.
30	*Get MS-DOS version number*. Returns: AL = major version number, AH = minor version number, BH = OEM serial number, BL:CX = 24-bit user serial number.
31	*Terminate and stay resident*. Terminate the current program or process, leaving part of itself in memory. Receives: AL = return code, and DX = requested number of paragraphs.
32	*Get MS-DOS drive parameter block*. Receives: DL = drive number. Returns: AL = status; DS:BX points to drive parameter block.
33	*Extended break checking*. Indicates whether or not MS-DOS is checking for Ctrl-Break.
34	*Get address of INDOS flag*. (Undocumented)
35	*Get interrupt vector*. Receives: AL = interrupt number. Returns: ES:BX = segment/offset of the interrupt handler.
36	*Get disk free space*. (FAT16 only) Receives: DL = drive number (0 = default, 1 = A, etc.). Returns: AX = sectors per cluster, or FFFFh if the drive number is invalid; BX = number of available clusters, CX = bytes per sector, and DX = clusters per drive.
37	*Get switch character*. (Undocumented)
38	*Get or set country information*.[a]
39	*Create subdirectory*. Receives: DS:DX points to an ASCIIZ string with the path and directory name. Returns: AX = error code if the Carry flag is set.

INT 21h Appendix (cont.)

3A	*Remove subdirectory.* Receives: DS:DX points to an ASCIIZ string with the path and directory name. Returns: AX = error code if the Carry flag is set.
3B	*Change current directory.* Receives: DS:DX points to an ASCIIZ string with the new directory path. Returns: AX = error code if the Carry flag is set.
3C	*Create or truncate file.* Create a new file or truncate an old file to zero bytes. Open the file for output. Receives: DS:DX points to an ASCIIZ string with the file name, and CX = file attribute. Returns: AX = error code if the Carry flag is set; otherwise AX = the new file handle.
3D	*Open existing file*. Open a file for input, output, or input-output. Receives: DS:DX points to an ASCIIZ string with the filename, and AL = the access code (0 = read, 1 = write, 2 = read/write). Returns: AX = error code if the Carry flag is set, otherwise AX = the new file handle.
3E	*Close file handle.* Close the file or device specified by a file handle. Receives: BX = file handle from previous open or create. Returns: If the Carry Flag is set, AX = error code.
3F	*Read from file or device.* Read a specified number of bytes from a file or device. Receives: BX = file handle, DS:DX points to an input buffer, and CX = number of bytes to read. Returns: If the Carry flag is set, AX = error code; otherwise, AX = number of bytes read.
40	*Write to file or device*. Write a specified number of bytes to a file or device. Receives: BX = file handle, DS:DX points to an output buffer, and CX = the number of bytes to write. Returns: If the Carry flag is set, AX = error code; otherwise, AX = number of bytes written.
41	*Delete file*. Remove a file from a specified directory. Receives: DS:DX points to an ASCIIZ string with the filename. Returns: AX = error code if the Carry flag is set.

INT 21h Appendix (cont.)

Table D-2 *(Continued)*

Function	Description
42	*Move file pointer*. Move the file read/write pointer according to a specified method. Receives: CX:DX = distance (bytes) to move the file pointer, AL = method code, BX = file handle. The method codes are as follows: 0 = move from beginning of file, 1 = move to the current location plus an offset, and 2 = move to the end of file plus an offset. Returns: AX = error code if the Carry flag is set.
43	*Get/Set file attribute*. Get or set the attribute of a file. Receives: DS:DX = pointer to an ASCIIZ path and filename, CX = attribute, and AL = function code (1 = set attribute, 0 = get attribute). Returns: AX = error code if the Carry flag is set.
44	*I/O control for devices*. Get or set device information associated with an open device handle, or send a control string to the device handle, or receive a control string from the device handle.
45	*Duplicate file handle*. Return a new file handle for a file that is currently open. Receives: BX = file handle. Returns: AX = error code if the Carry flag is set.
46	*Force duplicate file handle*. Force the handle in CX to refer to the same file at the same position as the handle in BX. Receives: BX = existing file handle and CX = second file handle. Returns: AX = error code if the Carry flag is set.
47	*Get current directory*. Get the full path name of the current directory. Receives: DS:SI points to a 64-byte area to hold the directory path, and DL = drive number. Returns: A buffer at DS:SI is filled with the path, and AX = error code if the Carry flag is set.
48	*Allocate memory*. Allocate a requested number of paragraphs of memory, measured in 16-byte blocks. Receives: BX = number of paragraphs requested. Returns: AX = segment of the allocated block and BX = size of the largest block available (in paragraphs), and AX = error code if the Carry flag is set.

INT 21h Appendix (cont.)

49	*Free allocated memory.* Free memory that was previously allocated by Function 48h. Receives: ES = segment of the block to be freed. Returns: AX = error code if the Carry flag is set.
4A	*Modify memory blocks.* Modify allocated memory blocks to contain a new block size. The block will shrink or grow. Receives: ES = segment of the block and BX = requested number of paragraphs. Returns: AX = error code if the Carry flag is set and BX = maximum number of available blocks.
4B	*Load or execute program.* Create a program segment prefix for another program, load it into memory, and execute it. Receives: DS:DX points to an ASCIIZ string with the drive, path, and filename of the program; ES:BX points to a parameter block and AL = function value. Function values in AL:0 = load and execute the program; 3 = load but do not execute (overlay program). Returns: AX = error code if the Carry flag is set.
4C	*Terminate process.* Usual way to terminate a program and return to either MS-DOS or a calling program. Receives: AL = 8-bit return code, which can be queried by DOS function 4Dh or by the ERRORLEVEL command in a batch file.
4D	*Get return code of process.* Get the return code of a process or program, generated by either function call 31h or function call 4Ch. Returns: AL = 8-bit code returned by the program, AH = type of exit generated: 0 = normal termination, 1 = terminated by CTRLBREAK, 2 = terminated by a critical device error, and 3 = terminated by a call to function call 31h.
4E	*Find first matching file.* Find the first filename that matches a given file specification. Receives: DS:DX points to an ASCIIZ drive, path, and file specification; CX = File attribute to be used when searching. Returns: AX = error code if the Carry flag is set; otherwise, the current DTA is filled with the filename, attribute, time, date, and size. DOS function call 1Ah (set DTA) is usually called before this function.

Common IRQ Assignments

- 0 System timer
- 1 Keyboard
- 2 Programmable Interrupt Controller
- 3 COM2 (serial)
- 4 COM1 (serial)
- 5 LPT2 (printer)
- 6 Floppy disk controller
- 7 LPT1 (printer)

Common IRQ Assignments

- 8 CMOS real-time clock
- 9 modem, video, network, sound, and USB controllers
- 10 (available)
- 11 (available)
- 12 mouse
- 13 Math coprocessor
- 14 Hard disk controller
- 15 (available)

Interrupt Control Instructions

- STI – set interrupt flag
  - enables external interrupts
  - always executed at beginning of an interrupt handler
- CLI – clear interrupt flag
  - disables external interrupts
  - used before critical code sections that cannot be interrupted
  - suspends the system timer

Writing a Custom Interrupt Handler

- Motivations
  - Change the behavior of an existing handler
  - Fix a bug in an existing handler
  - Improve system security by disabling certain keyboard commands
- What's Involved
  - Write a new handler
  - Load it into memory
  - Replace entry in interrupt vector table
  - Chain to existing interrupt hander (usually)

Get Interrupt Vector

- INT 21h Function 35h – Get interrupt vector
  - returns segment-offset addr of handler in ES:BX

```
.data
int9Save LABEL WORD
DWORD ?                 ; store old INT 9 address here
.code
mov ah,35h                      ; get interrupt vector
mov al,9                        ; for INT 9
int 21h                         ; call MS-DOS
mov int9Save,BX                 ; store the offset
mov [int9Save+2],ES             ; store the segment
```

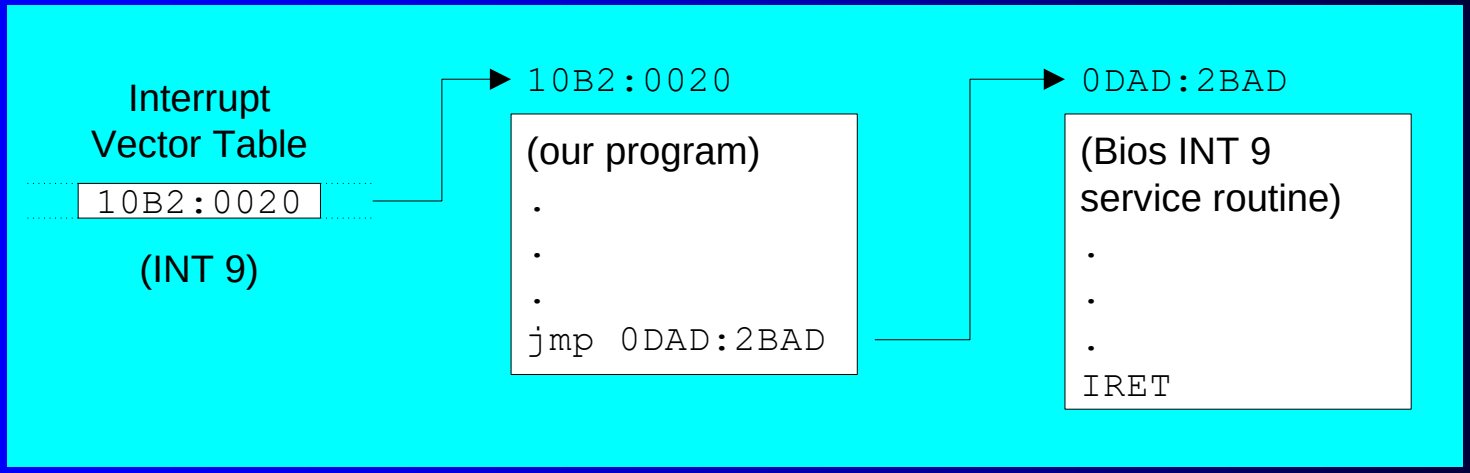
Set Interrupt Vector

- INT 21h Function 25h – Set interrupt vector
  - installs new interrupt handler, pointed to by DS:DX

```
mov ax,SEG kybd_rtn          ; keyboard handler
mov ds,ax                    ; segment
mov dx,OFFSET kybd_rtn       ; offset
mov ah,25h                   ; set Interrupt vector
mov al,9h                    ; for INT 9h
int 21h
.
.
kybd_rtn PROC                ; (new handler begins here)
```

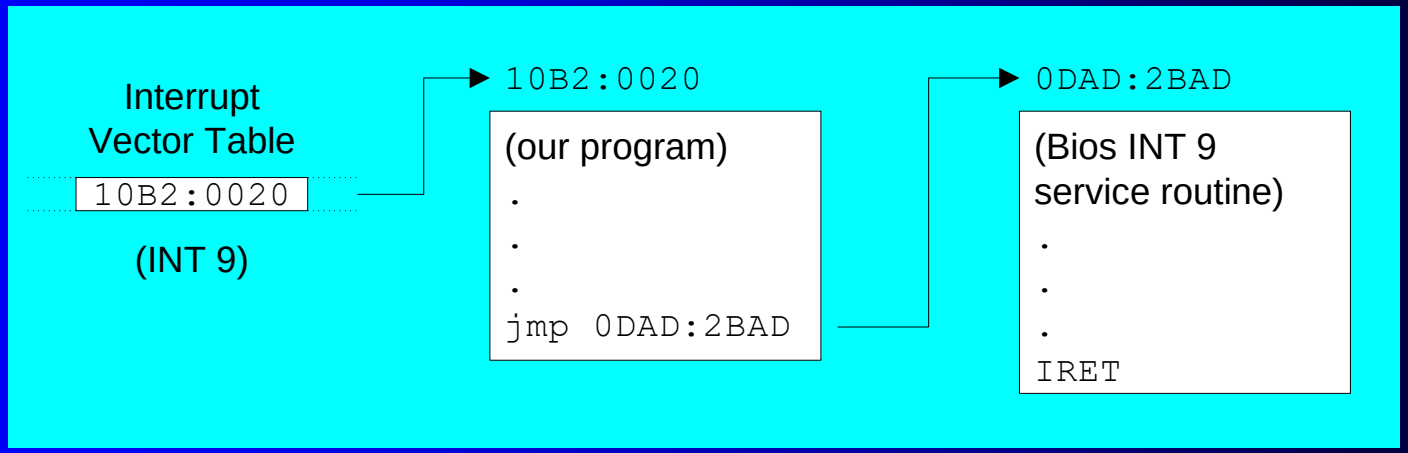
Keyboard Processing Steps

1. Key pressed, byte sent by hardward to keyboard port
2. 8259 controller interrupts the CPU, passing it the interrupt number
3. CPU looks up interrupt vector table entry 9h, branches to the address found there



Keyboard Processing Steps

4. Our handler executes, intercepting the byte sent by the keyboard
5. Our handler jumps to the regular INT 9 handler
6. The INT 9h handler finishes and returns
7. System continues normal processing



Ctrl-Break Handler Example

Ctrl-Break Handler Example

If Ctrl-Break is pressed by the user when an MS-DOS program is waiting for input, control passes to the default INT 23h interrupt handler procedure. The default Ctrl-Break handler terminates the currently running program. This can leave the current program in an unstable state because files might be left open, memory not released, and so on. It is possible, however, to substitute your own code into the INT 23h handler and prevent the program from halting. The following program installs a simple Ctrl-Break handler:

```
TITLE Control-Break Handler              (Ctrlbrk.asm)

; This program installs its own Ctrl-Break handler and
; prevents the user from using Ctrl-Break (or Ctrl-C)
; to halt the program. The program inputs and echoes
; keystrokes until the Esc key is pressed.

INCLUDE Irvine16.inc

.data
breakMsg BYTE "BREAK",0
msg  BYTE "Ctrl-Break demonstration."
     BYTE  0dh,0ah
     BYTE "This program disables Ctrl-Break (Ctrl-C). Press any"
     BYTE  0dh,0ah
     BYTE "keys to continue, or press ESC to end the program."
     BYTE  0dh,0ah,0
```

Ctrl-Break Handler Example

```
.code
main PROC
     mov    ax,@data
     mov    ds,ax

     mov    dx,OFFSET msg          ; display greeting message
     call   Writestring

install_handler:
     push   ds                     ; save DS
     mov    ax,@code               ; initialize DS to code segment
     mov    ds,ax
     mov    ah,25h                 ; set interrupt vector
     mov    al,23h                 ; for interrupt 23h
     mov    dx,OFFSET break_handler
     int    21h
     pop    ds                     ; restore DS

L1:  mov    ah,1                   ; wait for a key, echo it
     int    21h
     cmp    al,1Bh                 ; ESC pressed?
     jnz    L1                     ; no: continue

     exit
main ENDP

; The following procedure executes when Ctrl-Break is
; pressed. All registers must be preserved.

break_handler PROC

     push   ax
     push   dx
     mov    dx,OFFSET breakMsg
     call   WriteString
     pop    dx
     pop    ax
     iret
break_handler ENDP
END main
```

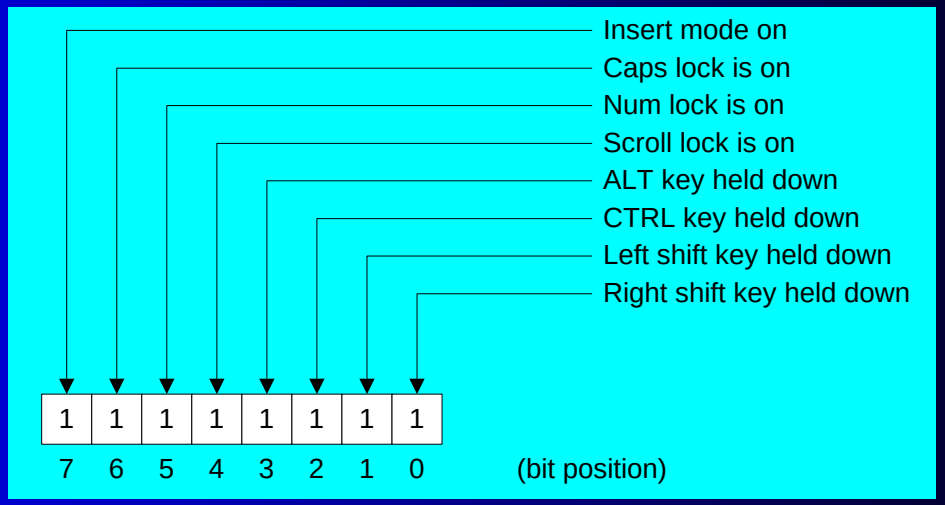
Terminate and Stay Resident Programs

- (TSR): Installed in memory, stays there until removed
  - by a removal program, or by rebooting
- Keyboard example
  - replace the INT 9 vector so it points to our own handler
  - check, or filter certain keystroke combinations, using our handler
  - forward-chain to the existing INT 9 handler to do normal keyboard processing

The No_Reset Program (1 of 6)

- Inspects each incoming key
- If the Del key is received,
  - checks for the Ctrl and Alt keys
  - permits a system reset only if the Right shift key is also held down

The keyboard status byte indicates the current state of special keys:



The No_Reset Program (2 of 6)

```
.code
    rt_shift    EQU 01h         ; Right shift key: bit 0
    ctrl_key    EQU 04h         ; CTRL key: bit 2
    alt_key     EQU 08h         ; ALT key: bit 3
    del_key     EQU 53h         ; scan code for DEL key
    kybd_port   EQU 60h         ; keyboard input port
```

The No_Reset Program (3 of 6)

```
int9_handler PROC FAR
    sti                         ; enable hardware interrupts
    pushf                       ; save regs & flags
    push  es
    push  ax
    push  di
```

The No_Reset Program (4 of 6)

- Locate the keyboard flag byte and copy into AH:

```
L1: mov   ax,40h              ; DOS data segment is at 40h
    mov   es,ax
    mov   di,17h              ; location of keyboard flag
    mov   ah,es:[di]          ; copy keyboard flag into AH
```

- Check to see if the Ctrl and Alt keys are held down:

```
L2: test  ah,ctrl_key         ; Ctrl key held down?
    jz    L5                  ; no: exit
    test  ah,alt_key          ; ALT key held down?
    jz    L5                  ; no: exit
```

The No_Reset Program (5 of 6)

- Test for the Del and Right shift keys:

```
L3: in    al,kybd_port        ; read keyboard port
    cmp   al,del_key          ; Del key pressed?
    jne   L5                  ; no: exit
    test  ah,rt_shift         ; right shift key pressed?
    jnz   L5                  ; yes: allow system reset
```

- Turn off the Ctrl key and write the keyboard flag byte back to memory:

```
L4: and   ah,NOT ctrl_key     ; turn off bit for CTRL
    mov   es:[di],ah          ; store keyboard_flag
```

The No_Reset Program (6 of 6)

- Pop the flags and registers off the stack and execute a far jump to the existing BIOS INT 9h routine:

```
L5: pop   di                          ; restore regs & flags
    pop   ax
    pop   es
    popf
    jmp   cs:[old_interrupt9]         ; jump to INT 9 routine
```